

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI -602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2
DEIGN AND CONNECTIVITY

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	Tool 1-DEIGN AND CONNECTIVITY
Weightage:		Total Marks:	40 Marks

CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)

Student Name:	Tharun V	Reg. No.:	192424050
Section / Batch:		Date of Submission:	
Faculty In-charge:	Dr. B.SHYAMALA GOWRI		

Project Mode: Individual Submission

Code of Conduct

Tharun V / 192424050 (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Tharun V

Requirement Analysis (5 Marks)	ER Diagram Design (10 Marks)	Table Design & Normalization (5 Marks)	Database Connectivity using Python (20 Marks)	CRUD Operations (5 Marks)	(35 Total Marks)

Scenario 1: Hospital Patient Record Management System

Requirement Analysis

A multi-specialty hospital receives thousands of patients every month. Automating patient records helps track patient history, doctor consultations, prescriptions, laboratory tests, and billing information efficiently. Key functional requirements include:

- **Patient Management:** Register and retain patient demographic details and contact records.
- **Doctor Management:** Maintain details regarding doctor specializations and availability.
- **Appointments & Consultations:** Track consultations between patients and doctors to monitor doctor workloads.
- **Prescriptions & Lab Diagnostic Tests:** Map specific prescriptions and lab reports directly to patient visits.
- **Billing Management:** Generate accurate financial invoices for hospital treatments and tests.

ER Diagram Design

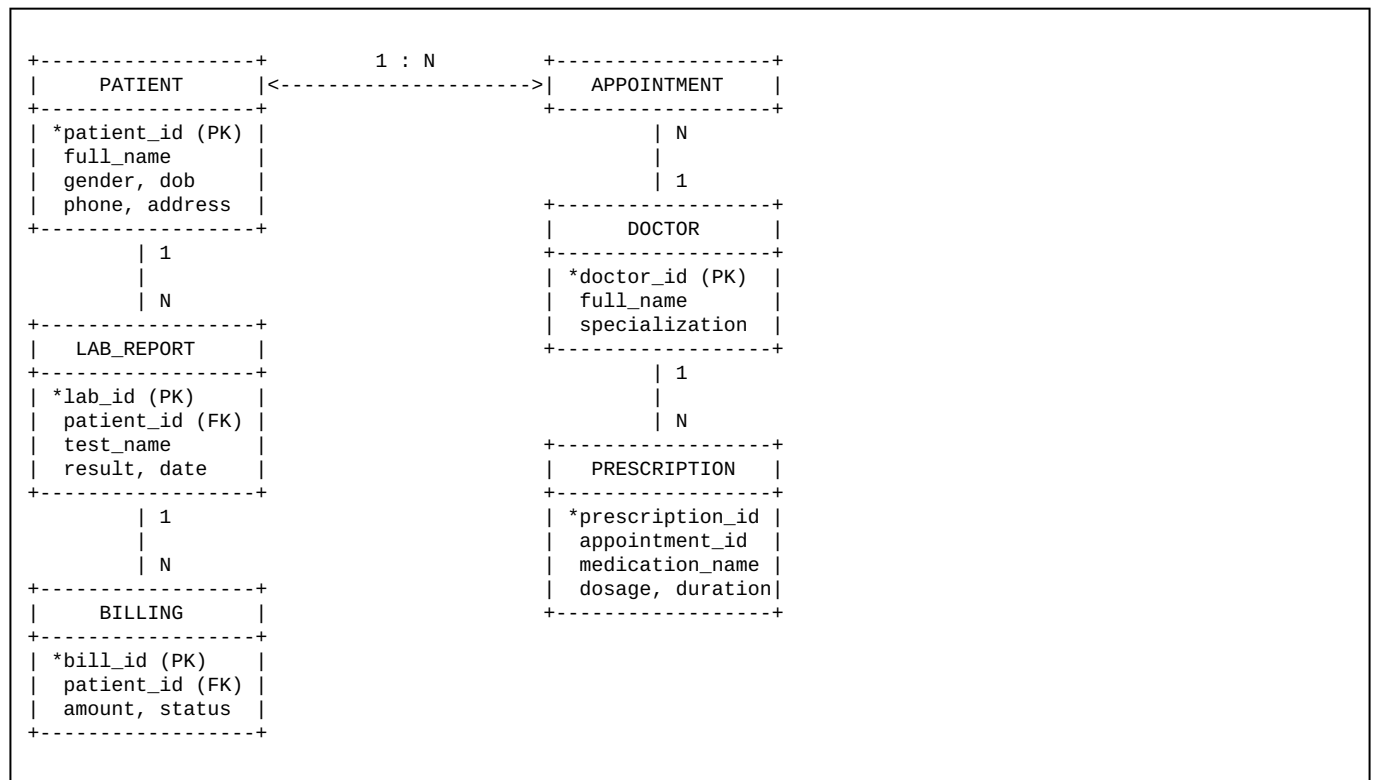


Table Design & Normalization (Up to 3NF)

Table Name	Primary Key	Foreign Keys	Attributes
Patients	patient_id	None	first_name, last_name, dob, gender, phone, address
Doctors	doctor_id	None	first_name, last_name, specialization, phone
Appointments	appointment_id	patient_id, doctor_id	appointment_date, diagnosis
Prescriptions	prescription_id	appointment_id	medication_name, dosage
LabTests	lab_id	patient_id	test_name, result, test_date
Billing	bill_id	patient_id	total_amount, payment_status

Database Connectivity using Python & CRUD Operations

```

import sqlite3

# Database Connectivity
conn = sqlite3.connect("hospital_management.db")
cursor = conn.cursor()
cursor.execute("PRAGMA foreign_keys = ON;")

# Schema Creation
cursor.executescript('''
CREATE TABLE IF NOT EXISTS Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    dob DATE NOT NULL,
    gender TEXT NOT NULL,
    phone TEXT UNIQUE NOT NULL
);

CREATE TABLE IF NOT EXISTS Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    specialization TEXT NOT NULL,
    phone TEXT NOT NULL
);
''')
    
```

```

CREATE TABLE IF NOT EXISTS Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    patient_id INTEGER NOT NULL,
    doctor_id INTEGER NOT NULL,
    appointment_date DATETIME NOT NULL,
    diagnosis TEXT,
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
);

CREATE TABLE IF NOT EXISTS Prescriptions (
    prescription_id INTEGER PRIMARY KEY AUTOINCREMENT,
    appointment_id INTEGER NOT NULL,
    medication_name TEXT NOT NULL,
    dosage TEXT NOT NULL,
    FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id)
);
'''
conn.commit()

# CRUD: CREATE
cursor.execute("INSERT INTO Doctors (first_name, last_name, specialization, phone) VALUES ('Rajesh', 'Kumar',
'Cardiology', '9876543210')")
cursor.execute("INSERT INTO Patients (first_name, last_name, dob, gender, phone) VALUES ('Tharun', 'V',
'2003-04-12', 'Male', '9123456789')")
cursor.execute("INSERT INTO Appointments (patient_id, doctor_id, appointment_date, diagnosis) VALUES (1, 1,
'2026-08-01 10:00:00', 'Hypertension')")
conn.commit()

# CRUD: READ
cursor.execute('''
    SELECT p.first_name, d.first_name, a.appointment_date, a.diagnosis
    FROM Appointments a
    JOIN Patients p ON a.patient_id = p.patient_id
    JOIN Doctors d ON a.doctor_id = d.doctor_id
''')
print("Consultations:", cursor.fetchall())

# CRUD: UPDATE
cursor.execute("UPDATE Patients SET phone = '9998887770' WHERE patient_id = 1")
conn.commit()

conn.close()

```

Scenario 2: E-Commerce Online Shopping Platform

Requirement Analysis

An online shopping platform selling products across categories requires dynamic cataloging, automated stock updating, payment tracking, and analytics reporting:

- **Customer Accounts:** Manage profile credentials and contact data.
- **Inventory Control:** Maintain stock quantities and update inventory levels after each completed transaction.
- **Order & Payment Processing:** Support multi-product orders with item breakdowns and payment status.
- **Reviews & Reporting:** Store customer feedback and generate monthly revenue and sales performance reports.

ER Diagram Design

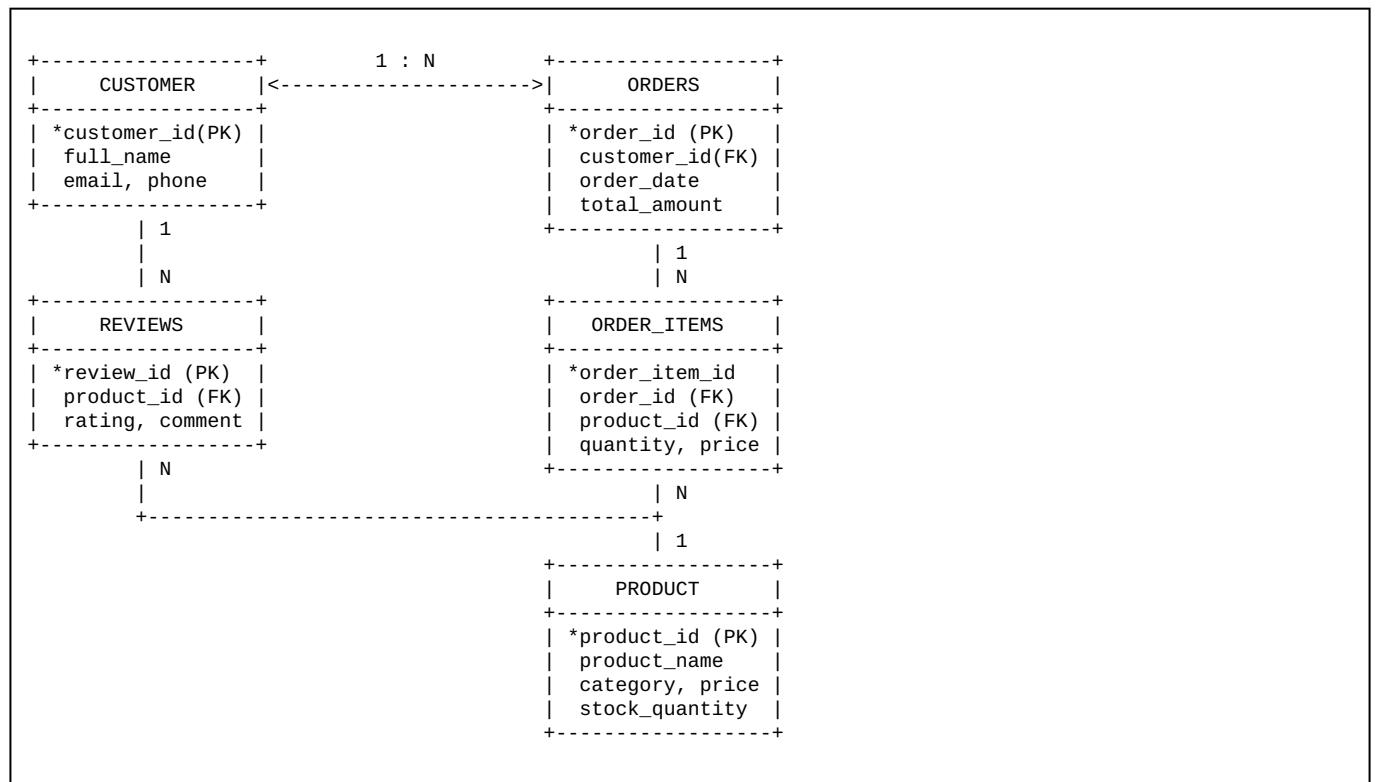


Table Design & Normalization (Up to 3NF)

Table Name	Primary Key	Foreign Keys	Attributes
Customers	customer_id	None	full_name, email, phone
Products	product_id	None	product_name, category, price, stock_quantity
Orders	order_id	customer_id	order_date, total_amount
OrderItems	order_item_id	order_id, product_id	quantity, unit_price
Reviews	review_id	customer_id, product_id	rating, comment

Database Connectivity using Python & CRUD Operations

```

import sqlite3

# Database Connection
conn = sqlite3.connect("ecommerce_platform.db")
cursor = conn.cursor()
cursor.execute("PRAGMA foreign_keys = ON;")

# Table Creation
cursor.executescript('''
CREATE TABLE IF NOT EXISTS Customers (
    customer_id INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    phone TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    category TEXT NOT NULL,
    price REAL NOT NULL,
    stock_quantity INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER NOT NULL,
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  
```

```

        total_amount REAL DEFAULT 0.0,
        FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
    );

CREATE TABLE IF NOT EXISTS OrderItems (
    order_item_id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    unit_price REAL NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
'''
conn.commit()

# Populate Initial Data
cursor.execute("INSERT INTO Customers (full_name, email, phone) VALUES ('Tharun V', 'tharun@example.com',
'9876543210')")
cursor.execute("INSERT INTO Products (product_name, category, price, stock_quantity) VALUES ('Wireless Mouse',
'Electronics', 1200.00, 50)")
conn.commit()

# Order Placement with Stock Decrement
def place_order(customer_id, product_id, quantity):
    cursor.execute("SELECT price, stock_quantity FROM Products WHERE product_id = ?", (product_id,))
    prod = cursor.fetchone()
    if prod and prod[1] >= quantity:
        price = prod[0]
        total_price = price * quantity
        cursor.execute("INSERT INTO Orders (customer_id, total_amount) VALUES (?, ?)", (customer_id,
total_price))
        order_id = cursor.lastrowid
        cursor.execute("INSERT INTO OrderItems (order_id, product_id, quantity, unit_price) VALUES
(?, ?, ?, ?)", (order_id, product_id, quantity, price))
        cursor.execute("UPDATE Products SET stock_quantity = stock_quantity - ? WHERE product_id = ?",
(quantity, product_id))
        conn.commit()
        print(f"Order #{order_id} placed successfully.")

place_order(1, 1, 2)

conn.close()

```